Linux Fest 2026

WebRTC for Those Who Know TCP/UDP



Who this talk is for

You already know

TCP: reliable, ordered byte stream

UDP: unreliable, unordered datagrams

Ports, sockets, client and server

Not assumed

NAT, STUN, TURN

TLS and certificates

Codecs, RTP, SDP

Anything WebRTC







What is WebRTC?

A protocol

Rules for two agents to set up secure, two-way, real-time communication.

An agent is a browser, an app or a server.

An API

The functions a developer calls to use the protocol.

Specified only for JavaScript.

Real-time: audio, video and data between peers with very low delay.



Four phases of every call



Signaling
Exchange addresses, codecs, keys info.

Connecting
Find a network path.

Securing
Encrypt and verify.

Communicating
Send media and data.

The big problem we solve today

The situation

Alice wants to talk to Bob live, from a browser.

The internet in between is hostile: routers, firewalls, delay, loss, strangers.

What we need

Find a path through the routers.

Agree on how to talk. Keep it private.

Keep sound and picture smooth.

Our plan

Every part follows the same pattern: 1 the problem 2 what we need 3 numbered points that solve it



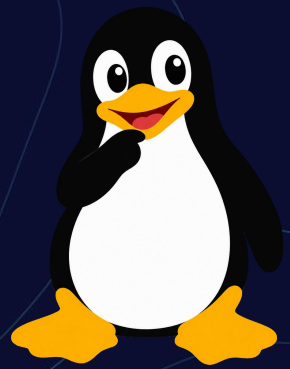
Why not just TCP or UDP?

TCP

+ Reliable, ordered delivery
+ Congestion control built in
- A lost packet stalls everything behind it
- Late audio is worse than lost audio

UDP

+ Low latency, no waiting
- No reliability or ordering
- No congestion control
- No NAT traversal, encryption or codecs



Five problems, five answers

1	Find a path behind NATs	→	ICE, STUN, TURN
2	Agree on who and what	→	Signaling and SDP
3	Stay private on an open network	→	DTLS and SRTP
4	Carry live media	→	RTP, RTCP, congestion control
5	Carry arbitrary data	→	SCTP data channels



Real time means a latency budget



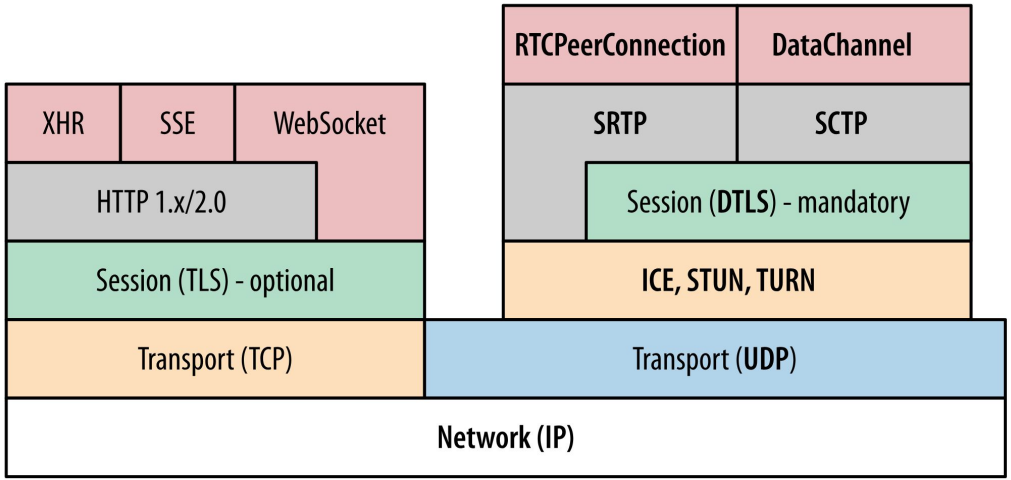
Under about 150 ms one way

Feels like talking in the same room.

Over about 400 ms

People talk over each other. Conversation breaks down.

TCP stack vs WebRTC stack



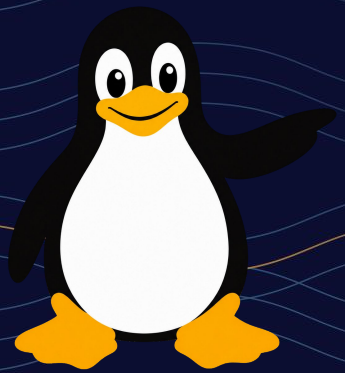
Read it bottom up

Both sit on IP.

Web: TCP, optional TLS, then HTTP or WebSocket.

WebRTC: UDP, then ICE, STUN and TURN, then mandatory DTLS, then SRTP and SCTP.

Part 1: Connecting and signaling



The problem: peers cannot find each other

The situation

Alice and Bob each sit behind a home router.

The router is a receptionist: it lets calls out, but blocks unexpected calls in.

Neither knows the other's real address.

What we need

1. Learn my own public address.
2. Open a door through both routers.
3. A fallback if no door can be opened.

Our plan

NAT, STUN and TURN, then ICE picks a path Then signaling and SDP, to hand over the details



What is NAT?

Why it exists

IPv4 has about 4.3 billion addresses, far fewer than devices.

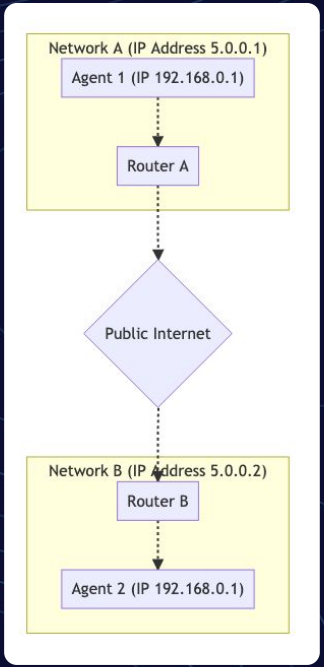
What it does

The router rewrites private ip:port to its public ip:port and remembers the mapping.

Side effect

Outbound opens a door. Unsolicited inbound is dropped.

Two networks, one internet



Far apart, same private address

Different networks, same private address 192.168.0.1.

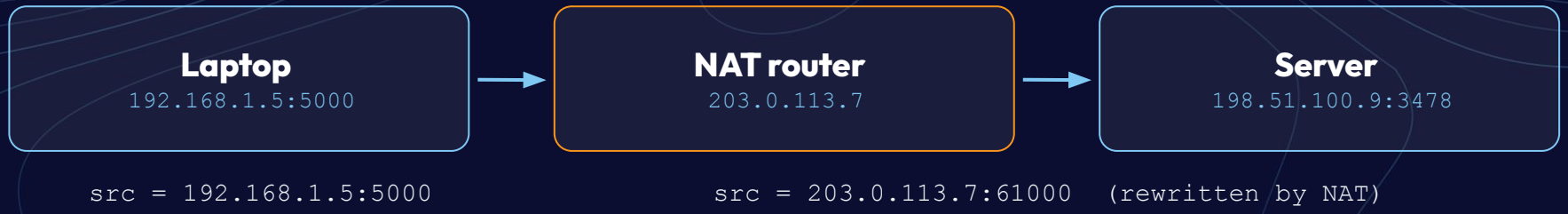
Private addresses mean nothing on the public internet.

A NAT mapping is a door the router opens

Agent 1 uses port 7000. The router binds 192.168.0.1:7000 to 5.0.0.1:7000.

Agent 2 can now send to 5.0.0.1:7000 and reach Agent 1.

How a NAT mapping works



NAT table entry

`192.168.1.5:5000 <-> 203.0.113.7:61000 (talking to 198.51.100.9:3478)`

Replies to 61000 are translated back. Anything else is dropped.

Not all NATs are equal

Type	Who may send in	Hole punching
Full cone	Anyone, once a mapping exists	Works
Restricted cone	Only IPs you already sent to	Works
Port-restricted	Only ip:port you already sent to	Works
Symmetric	Different mapping per destination	Usually fails, use relay



NAT types by example

Laptop sent to S1 port **3478** and got port 61000. Stranger: 192.0.2.50. Cone types keep one port.

Full cone

Same :61000 for every destination.
Stranger 192.0.2.50 to :61000: delivered.

Restricted cone

Same :61000 for every destination.
S1 port 4000: delivered. Stranger: dropped.

Port-restricted cone

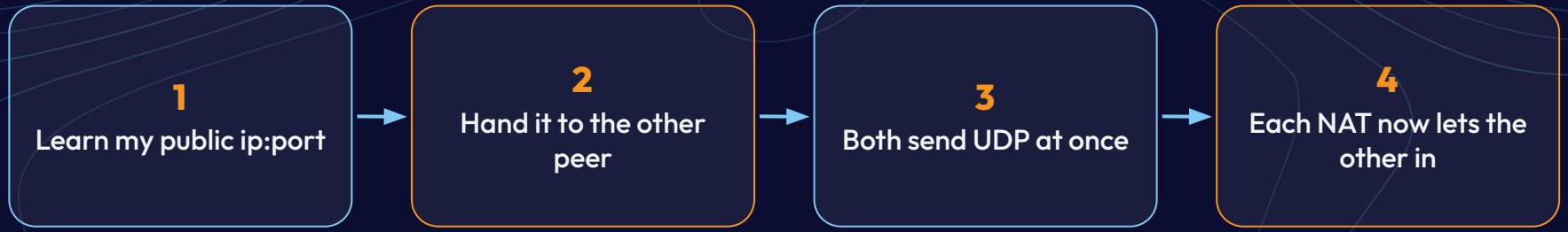
Same :61000 for every destination.
S1 port 3478: delivered. Port 4000: dropped.

Symmetric

New port per destination: :61000 for S1, :61001 for B.
STUN said 61000, so B cannot reach it. Use TURN.



Hole punching



Each outgoing packet opens a mapping in its own NAT.

The other side's packet then matches that mapping and gets through.

STUN, my public address

How it works

1. Send a Binding Request over UDP.
2. The server replies with the ip:port it saw.
3. That is your server-reflexive address.

Properties

Tiny, stateless, cheap.

No media passes through it.

Address unusable behind symmetric NAT.

```
You: "what do you see as my address?"   STUN: "203.0.113.7:61000"
```



STUN in action: Alice calls Bob



1 Ask STUN
Each learns its public ip:port from the server.

2 Swap addresses
Alice gives Bob 203.0.113.7:61000. Bob gives Alice 192.0.2.77:40000.

3 Send to each other
Both send UDP at once. Each NAT opens a door for the other.

4 Talk directly
Media flows peer to peer. STUN is no longer involved.

TURN: why we need a relay

The problem: no direct path

Alice is behind a symmetric NAT. STUN saw her as 203.0.113.7:61000.

To talk to Bob, her NAT opens a new port: 61001.

Bob sends to 61000, the port he was told. It is only for STUN: dropped.

Alice sends from 61001. Bob's NAT only expects 61000: dropped.

An office firewall that blocks all UDP fails the same way.

The fix: both talk to a relay

Alice and Bob each send out to TURN server 198.51.100.20. Outgoing traffic always passes a NAT.

TURN gives Alice a relay address: 198.51.100.20:50000.

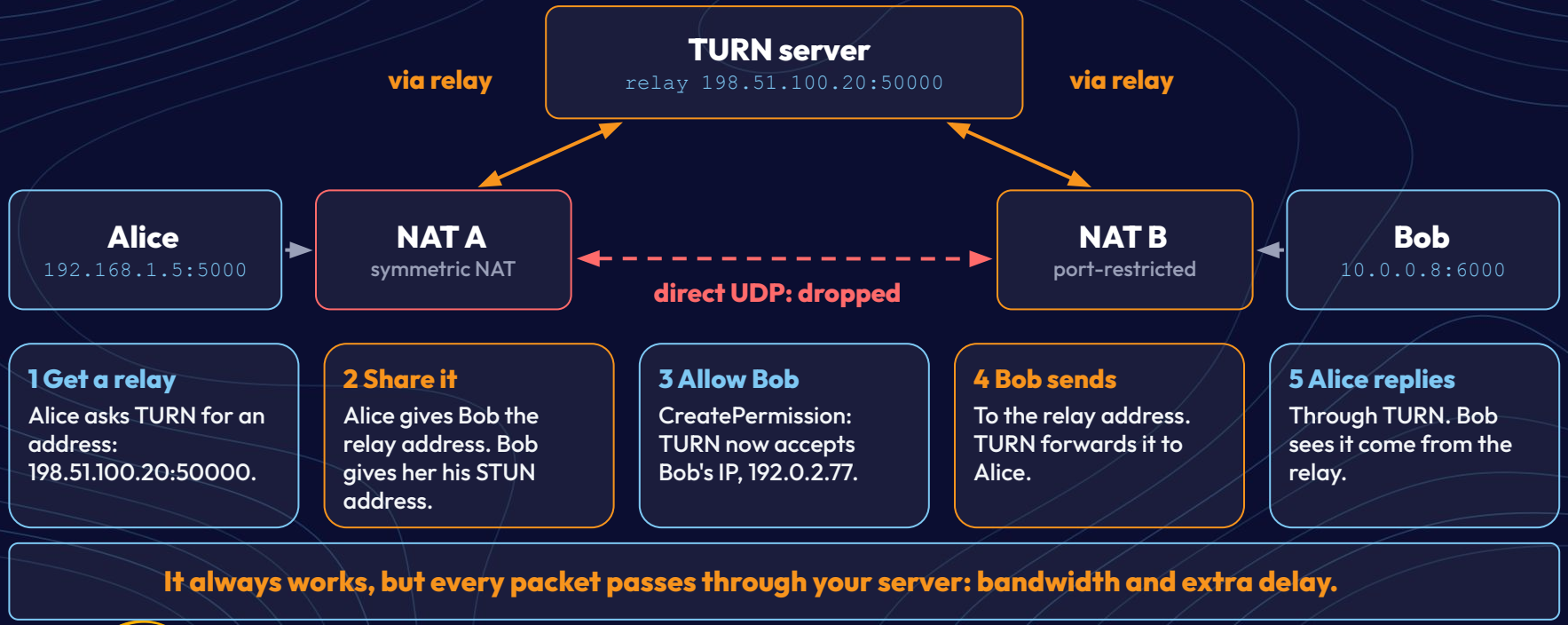
Bob sends there; the server forwards to Alice, and back.

Works over UDP, TCP or TLS on port 443, so firewalls let it through.

Cost: every packet passes through your server: bandwidth and extra delay. So TURN is the last resort.



TURN in action: Alice calls Bob



ICE: which address should Bob use?

The problem

Someone must ask STUN for Alice's public address and TURN for a relay address.

Then which one should Bob use: her own, STUN or relay address? Some fail, some are slow.

Someone must test them all, and do it again if Wi-Fi drops mid-call.

The answer: ICE

Interactive Connectivity Establishment does all of it.

Each peer runs an ICE agent in its browser or app. It asks STUN and TURN, tests every path, and keeps the best one.



ICE in five stages. The next slides take them one at a time.

ICE: gather candidates

A candidate is an ip:port where the peer might reach me. Alice has three:

host

From her own network card.

`192.168.1.5:5000`

Works only inside the same network.

server reflexive

From the STUN server.

`203.0.113.7:61000`

Works if her NAT lets it.

relay

From the TURN server.

`198.51.100.20:50000`

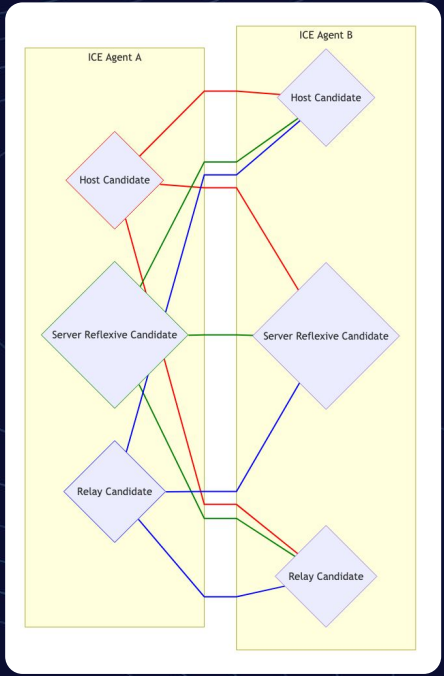
Always works, but costs the most.

Priority: host, then server reflexive, then relay. Cheapest and fastest first.

A fourth kind, peer reflexive, is discovered later, during the checks.



ICE: exchange and pair



Exchange

Each peer sends its list to the other, often one candidate at a time as it is found: trickle ICE. How it travels is the next part, signaling.

Pair

Every local candidate meets every remote one: 3 x 3 = 9 pairs. Each line in the picture is one pair.

Sort

Pairs are sorted by priority, so the cheapest direct paths are tried first.

ICE: check, pick and keep

1
Check
The ICE agent sends a small test packet on every pair. A reply means that path works.

→

2
Pick
The best pair that works wins. Media and data use only that path.

→

3
Keep
It keeps checking during the call, and looks again if the network changes.

Alice and Bob end up on relay to server reflexive:

```
Alice  <->  TURN 198.51.100.20:50000  <->  Bob 192.0.2.77:40000
```



The problem: how do they meet first?

The situation

We have tools to find a path, but peers must still tell each other their addresses, codecs and key fingerprint.

No direct path exists yet to send that. Chicken and egg.

What we need

A go-between both can reach: a server.

A shared format for the information.

A polite order: ask, then answer.

Our plan

1 Signaling 2 Flow 3 SDP basics 4 Reading SDP 5 Structure fields 6 Codec, ICE, security fields 7 Offer and answer



Signaling, meeting first

What must be shared

Candidate addresses, codecs, security fingerprint.

Why a server

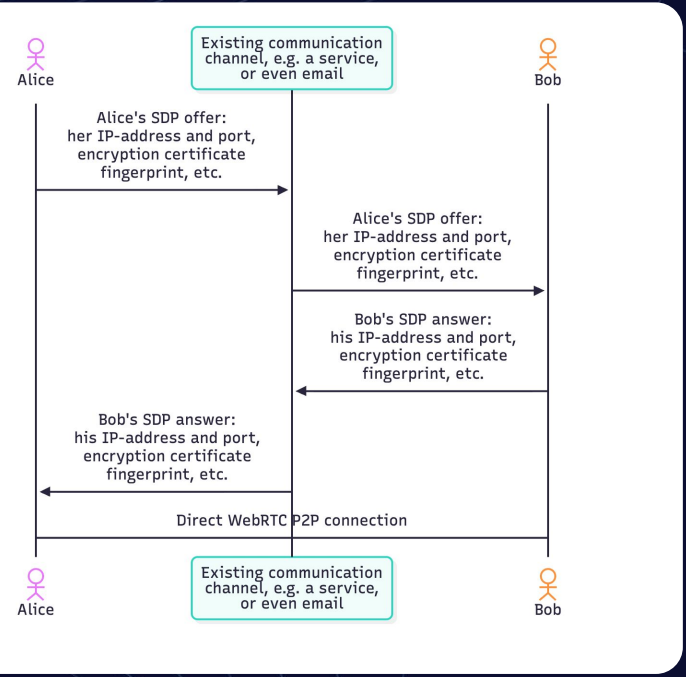
No direct path yet, so use a server both can reach.

Which protocol

Not defined by WebRTC. WebSocket, HTTP, SIP: your choice.



The signaling flow



Alice offers, Bob answers

Each SDP carries addresses, codecs and the certificate fingerprint.

Any channel works

The middle box can be a web service, a chat app, even email.

After the exchange the peers connect directly.

SDP, the call description

Format

Plain text, one key=value per line.

Describes

Media, codecs, how to reach me, how to secure it.

In practice

The browser writes and reads it. You carry it between peers.

```
m=audio 9 UDP/TLS/RTP/SAVPF 111
```



Reading an SDP

```
v=0
m=audio 9 UDP/TLS/RTP/SAVPF 111
a=rtpmap:111 opus/48000/2
a=ice-ufrag:Xk3f
a=ice-pwd:9aZ...
a=fingerprint:sha-256 AB:CD:...
a=setup:actpass
m=application 9 UDP/DTLS/SCTP
  webrtc-datachannel
```

Codecs

What each side can send and receive

ICE credentials

Who may send connectivity checks

DTLS fingerprint

Hash of the peer certificate



SDP fields, structure

Field	Meaning
v=0	Version of SDP, always 0
o=- 4611 2 IN IP4 127.0.0.1	Session id and version, bumped on each change
s=-	Session name, unused in WebRTC
t=0 0	Timing, always 0 0 (no limit)
a=group:BUNDLE 0 1	All media share one transport, one UDP port
m=audio 9 UDP/TLS/RTP/SAVPF 111	Media section: kind, dummy port, transport, payload IDs

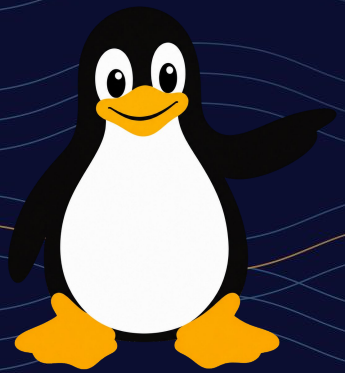


SDP fields, codec, ICE, security

Field	Meaning
a=rtpmap:111 opus/48000/2	Payload 111 is Opus, 48 kHz, 2 channels
a=fmtp:111 useinbandfec=1	Codec settings, e.g. in-band FEC
a=sendrecv	Direction: sendrecv, sendonly, recvonly, inactive
a=ice-ufrag / a=ice-pwd	Username and password for ICE checks
a=candidate:...	An address to try: host, srflx or relay
a=fingerprint:sha-256 AB:CD	Hash of my DTLS certificate
a=setup:actpass	DTLS role: offer actpass, answer active or passive



Part 2: Securing



The problem: strangers can read the call

The situation

UDP packets cross routers owned by strangers.

Anyone on the path can read or alter them.

Peers have no certificate from a trusted authority.

What we need

Encryption that works over lossy UDP.

A way to check we talk to the right peer.

No unencrypted mode at all.

Our plan

1 What we get 2 Keys 3 Certificate 4 Fingerprint 5 Overview 6 Hello 7-10 Shared secret 11 Two keys 12-13 Handshake 14 Keys 15 Safe



What security do we get?

Private

Nobody on the way can read the call.

Untouched

Nobody can change or add packets.

The right person

You really talk to Bob, not someone pretending.

Always on, with one condition

There is no way to turn encryption off in WebRTC.

But the SDP must reach Bob unchanged, because it carries the information used to recognise him. So send signaling over HTTPS.





Overview: DTLS, then SRTP

1. ICE path ready
2. DTLS handshake: 6 flights
3. Both compute the same keys
4. Check Bob's certificate against the SDP
5. Export keys to SRTP

DTLS

TLS, the protocol behind HTTPS, adapted to UDP.

Lost handshake messages are resent on a timer.

Afterwards it carries the data channels.

SRTP

Encrypts and signs each RTP packet.

No handshake of its own: it gets its keys from DTLS.

RTCP is protected the same way: SRTCP.



One certificate, two key pairs

	Public part: sent to Alice	Private part: never leaves Bob
Identity key pair long-lived	Public key, 65 bytes, put inside the certificate	Private key, 32 bytes Job: SIGN, to prove "this is Bob"
Certificate exactly one	The whole 379-byte file: name, dates, public key, signature	Nothing. A certificate has no private part.
Fresh key pair new every handshake	Public half, 65 bytes, sent in ServerKeyExchange	Private half, 32 bytes Job: MAKE THE SECRET, then deleted

Like a passport (certificate), your face (private key) and a padlock made for one conversation (fresh key).



Signature or fingerprint?

	Signature	Fingerprint
Value	`30:45:02:20:73:12:a8:9e:...`	`4F:76:F2:E0:AA:C8:34:4D:...`
Size	71 bytes	32 bytes
Computed over	Content only (part 1)	The whole 379-byte certificate
How	Hash, then Bob's identity key	SHA-256 only, no key
Travels in	The certificate, in the handshake	The SDP: a=fingerprint:sha-256

Trust comes from the SDP

Bob's fingerprint reached Alice through your HTTPS signaling, where your app knows the user is Bob. The certificate in the handshake must hash to exactly that value.





Inside the certificate

1 Content

```
Subject:   CN = WebRTC
Issuer:    CN = WebRTC    <- same: self-signed
Valid:     30 days
Key:       04:dc:e1:d5:e5:98:12:52...
```

2 Algorithm: ecdsa-with-SHA256

3 Signature (71 bytes)

```
30:45:02:20:73:12:a8:9e:...:54:d8:5b:e6:45:a1
```

What the signature is

Hash part 1 with SHA-256, then seal that hash with Bob's identity private key.

Anyone checks it with the public key in part 1.

Self-signed: Bob's browser signed its own certificate.

```
openssl verify bob.crt
bob.crt: OK
Change one letter: WebRTC -> XebRTC
verification failed
```

What it proves, and whatnot

The file is intact and its maker holds the private key. Not that the maker is Bob.



Steps 1-3: hello and cookie

Message	Example content	Why
1 ClientHello **Alice to Bob**	`version: DTLS 1.2, ciphers, use_srtp` `random: 7f3a91c2... (32 bytes)`	Alice lists what she can do. Fresh random for this call.
2 HelloVerifyRequest **Bob to Alice**	`cookie: 9c41e07b...` `= hash(Bob's secret + Alice's ip:port)`	Bob stores nothing and sends nothing big yet.
3 ClientHello **Alice to Bob**	`same as message 1` `+ cookie: 9c41e07b...`	Bob recomputes the hash. It matches, so the address is real.

The problem: UDP addresses can be faked

Mallory sends a tiny ClientHello with a victim's address as sender.

Without a cookie, Bob sends his big flight (~1 KB) to the victim and keeps memory for it.

The cookie: prove you own the address

The cookie goes to the real address. Mallory never sees it, so she cannot send message 3.

Real Alice receives it and repeats her hello. Costs one round trip.

Step 4: Bob answers

Message	Example content	Why

`ServerHello`
Bob to Alice
	`cipher: ECDHE-ECDSA-AES128-GCM-SHA256`
	`use_srtp: SRTP_AES128_CM_HMAC_SHA1_80`
	`random: b28e44d0... (32 bytes)`
		Bob picks from Alice's lists and adds his own random.

`Certificate`	`379 bytes, self-signed, key 04:dc:e1:d5...`	Alice hashes it: 4F:76:F2... equals the SDP. Check 1 passed.

`ServerKeyExchange`	`fresh public half: 04:0b:f7:f8...`
	`signature by Bob's identity key,`
	`over both randoms + this fresh key`
		Check 2: Alice verifies it with the public key in Bob's certificate. The fresh key is Bob's.

`CertificateRequest`	`(no content worth showing)`	Alice must show a certificate too.
`ServerHelloDone`	`(empty)`	End of Bob's flight.

Mallory swaps the fresh key: signature fails. Mallory sends her own certificate: fingerprint differs.



Steps 5-6: Alice answers, both switch on

Message	Example content	Why
Certificate **Alice to Bob**	379 bytes, self-signed fingerprint CE:9F:0D... in her SDP	Bob hashes it and compares with Alice's SDP.
ClientKeyExchange **Alice to Bob**	fresh public half: 04:fd:b1:d9... (65 bytes)	Now both have both halves: both compute 78c9e35b...
CertificateVerify **Alice to Bob**	signature over messages so far made with Alice's identity key	Proves Alice owns her certificate.
ChangeCipherSpec **Alice to Bob**	(one byte)	Everything Alice sends after this is encrypted.
Finished **Alice to Bob**	ENCRYPTED: hash of the handshake	If anything earlier was changed, Bob's hash differs.
Finished **Bob to Alice**	ChangeCipherSpec, then ENCRYPTED: hash of the handshake	Bob switches too. Handshake done.



Fresh keys, step by step

Step	Alice	On the network	Bob
1 Make a pair	private a (kept secret) `public A = 04:fd:b1...`	nothing yet	private b (kept secret) `public B = 04:0b:f7...`
2 Send public	sends A in `ClientKeyExchange`	A and B: anyone can see them	sends B in `ServerKeyExchange, signed`
3 Combine	`a x B` = 78c9e35b...53dc4dbb	nothing sent	`b x A` = 78c9e35b...53dc4dbb
4 Delete	a is thrown away	the secret never crossed the network	b is thrown away

Why the same?

A = a x G, B = b x G, so a x B = a x b x G = b x A

Why safe?

Getting a back from A is practically impossible.



Why do both get the same secret?

Pretend "combine" is ordinary multiplication, and the public start is G = 7:

	Alice	Bob
Secret	`a = 6`	`b = 15`
Public	`A = 6 x 7 = 42`	`B = 15 x 7 = 105`
Combine	`6 x 105 = 6 x 15 x 7 = 630`	`15 x 42 = 15 x 6 x 7 = 630`

Same three ingredients

Both results are 6 x 15 x 7. Each side adds its own secret to the other's wrapped one. Order does not matter.

But plain multiplication is weak

A spy divides: 42 / 7 = 6, then 6 x 105 = 630.

We need: order does not matter AND no way back.





What is G, and where does it come from?

G is a fixed starting point

A point on the curve, written into the P-256 standard (NIST, around 2000).

Built into every crypto library: Chrome, Firefox, Android, iOS.

Public, never sent, never generated.

Peers only agree on the name

ClientHello: supported_groups: x25519, secp256r1, ...

ServerKeyExchange: named_curve: secp256r1 (2 bytes)

Both look up the same G in their own table.

The real G of P-256 (65 bytes: 04, then x, then y)

```
x  6b:17:d1:f2:e1:2c:42:47:f8:bc:e6:e5:63:a4:40:f2:77:03:7d:81:2d:eb:33:a0:f4:a1:39:45:d8:98:c2:96
y  4f:e3:42:e2:fe:1a:7f:9b:8e:e7:eb:4a:7c:0f:9e:16:2b:ce:33:57:6b:31:5e:ce:cb:b6:40:68:37:bf:51:f5
```

Same format as Alice's A = 04:fd:b1... and Bob's B = 04:0b:f7..., which are G combined a and b times.

Easy forward, impossible backward

Tiny curve, mod 1009, G = (10, 425)

```
A = 6 x G       = (319, 222)
B = 15 x G      = (364, 293)

Alice 6 x B     = (472, 242)
Bob  15 x A     = (472, 242)
90 x G          = (472, 242)
```

Order does not matter: 6 x 15 = 90.

Can we go back from A to a?

```
2G = (960,840)   3G = (265,924)
4G = (738,22)    5G = (853,384)
```

Points jump around: no pattern, no division. Only way: try 1, 2, 3, ... until you hit A.

Tiny curve, 1068 points: 6 tries.

	Forward: a to A	Backward: A to a
Real P-256, ~10^77 points	microseconds	~2^128 steps
Fastest supercomputer	microseconds	~1000 x age of universe

Identity key and fresh key: why both?

Identity key: SIGNS

Made with the certificate, lives the whole session.
Signs the certificate and Bob's fresh public half.
Never encrypts anything.

Fresh key: MAKES THE SECRET

New for every handshake.
Combined with the other side's half: 78c9e35b...
Deleted afterwards.

Without the identity key, Mallory in the middle swaps the fresh keys:



With it, Bob's fresh key arrives signed. Mallory cannot sign as Bob, and her own certificate does not match Bob's fingerprint.

From secret to keys

```
1  master = PRF( 78c9e35b..., "master secret", 7f3a91c2... + b28e44d0... )
```

= 6ac20a94... 48 bytes. Inputs: pre-master from the fresh keys, both randoms from the Hellos.

```
2  srtp = PRF( 6ac20a94..., "EXTRACTOR-dtls_srtp", 7f3a91c2... + b28e44d0... )
```

= 60 bytes, cut into four pieces:

Alice key
bytes 1-16
`afa16c7e59e4337c...`

Bob key
bytes 17-32
`edf969b92462c459...`

Alice salt
bytes 33-46
`6ddd9df7ddfabfbb...`

Bob salt
bytes 47-60
`11838dc5189ff31e...`

PRF: a blender built from SHA-256

Same inputs, same output on both sides. Change one input bit: totally different output. Any length you ask for.

Who uses which piece

Alice to Bob: Alice key + Alice salt. Bob decrypts with the same pair.
Bob to Alice: Bob key + Bob salt.





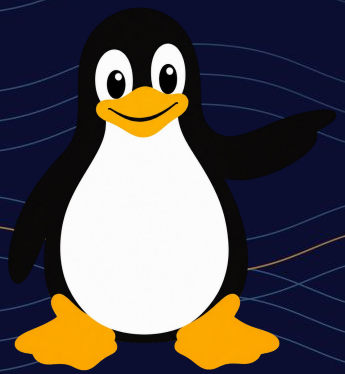
Why plaintext is safe

Mallory on the path reads every handshake message. Here is why that does not help her.

He reads the randoms	Not secret. They only make this call's keys unique.
He reads 04:0b... and 04:fd...	Without a private half he cannot compute 78c9e35b...
He reads the certificates	Public keys only. Private keys never leave the browsers.
He swaps Bob's fresh key	She cannot sign it: she does not have Bob's identity key.
He sends his own certificate	Hash is not 4F:76:F2... from the SDP. Call dropped.
He replays yesterday's messages	The randoms differ, so signatures and Finished fail.



Part 3: Real-time networking



The problem: an unreliable network

The situation

The path between Alice and Bob is shared, changing and imperfect.

Packets get lost, arrive unevenly, and too much traffic jams the path.

Late data is useless in a call.

What we need

Understand what can go wrong.

Ways to recover lost packets.

Smooth uneven arrival.

Notice congestion and adapt.

Our plan

1-2 Vocabulary 3 RTT 4 Jitter buffer 5-6 Loss 7 Congestion 8-10 Estimators 11 Probing 12 Pacer 13 Adapt



Vocabulary (1/2): capacity and delay

Term	What it means	Example
Bandwidth	How much data the path can carry per second. Shared, so it changes.	`home uplink: 10 Mbps`
Bitrate	How much data we choose to send per second.	`video 1.5 Mbps, audio 32 kbps`
Latency	Time for one packet to go from Alice to Bob. Also called one-way delay.	`40 ms`
RTT	Round trip time: there and back again.	`80 ms`

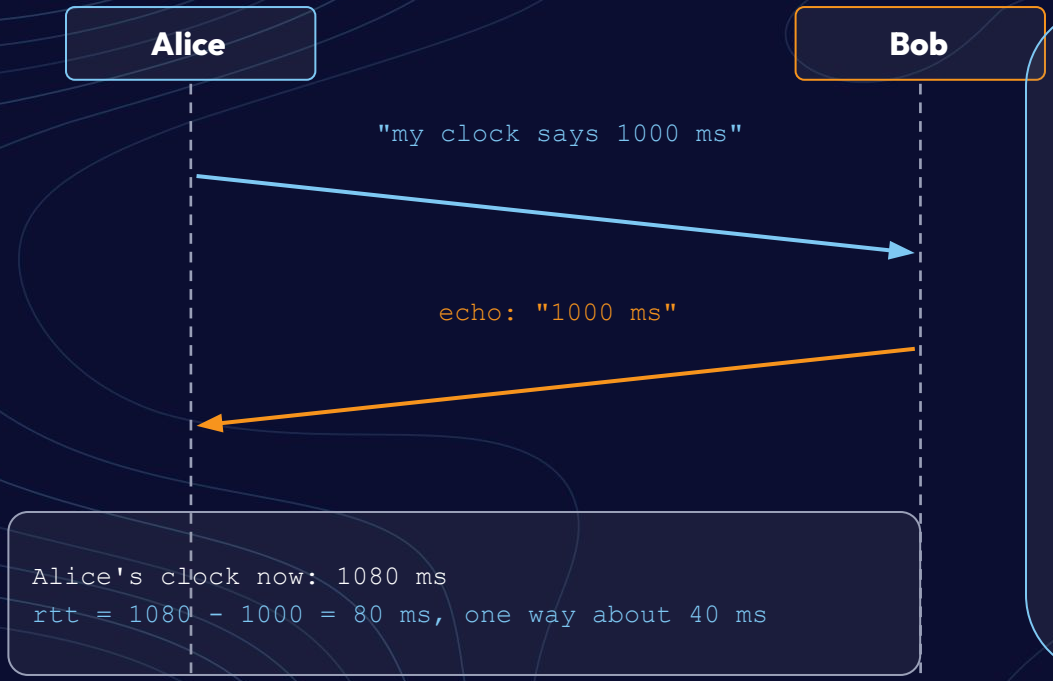


Vocabulary (2/2): what goes wrong

Term	What it means	Example
Jitter	Delay changes from packet to packet. Even spacing becomes uneven.	`sent every 20 ms, arrive 26, 13, 48, 4 ms apart`
Packet loss	Packets that never arrive.	`2 of 100 lost = 2%`
MTU	Largest packet the path carries. Bigger data is split.	`1500 bytes; RTP keeps ~1200`
Congestion	Sending more than the path can carry: queues grow, then packets drop.	`2 Mbps into a 1.5 Mbps link`



Measuring delay: round trip time



Why round trip?

Alice's and Bob's clocks do not agree, so "sent at, arrived at" cannot be compared.

With RTT only Alice's own clock is used.

rtt / 2 assumes both directions are equal. On mobile, upload is often slower, so it is only an estimate.

WebRTC does this with RTCP reports (media part).

Jitter buffer: an example

Alice sends a 20 ms audio packet every 20 ms. Simplified fixed playout schedule; real WebRTC buffers adapt.

	Packet 1	Packet 2	Packet 3	Packet 4
Sent at	0 ms	20 ms	40 ms	60 ms
Arrives at	45 ms	71 ms	84 ms	132 ms
Network delay	45 ms	51 ms	44 ms	72 ms
Change in network delay	baseline	+6 ms	-7 ms	+28 ms
No buffer: plays at	45 ms	71 ms	84 ms	132 ms
30 ms buffer: plays at	75 ms	95 ms	115 ms	135 ms

A jitter buffer delays playback to absorb variations in packet arrival times.

No buffer

Gaps of 26, 13, 48 ms between arrivals: choppy audio.

30 ms buffer

Plays every 20 ms. Packet 4: arrives 132, due 135.

20 ms buffer: too small

Packet 4 is due at 125 ms but arrives at 132 ms, 7 ms late. It misses its playback deadline.





Solving packet loss: ACK, SACK, NACK, FEC

Alice sends 10 packets:



	What it is	In our example	Good / bad
ACK	Acknowledgment: the receiver confirms every packet it got.	`Bob: ack 1, ack 2, ack 3, ack 4, then ack 4 again, again`	Repeated ack 4: 5 is missing.
SACK	Selective ACK: one message lists all ranges that arrived.	`Bob: "got 1-4 and 7-10"`	Sender sees the gap 5-6 at once.
NACK	Negative ACK: the receiver reports only what is missing.	`Bob: "NACK 5, 6"  Alice resends 5, 6`	Fewest messages. WebRTC uses it.
FEC	Forward error correction: sender adds repair data in advance.	`Alice also sends 11 = 5 XOR 6. Lost 5? 5 = 11 XOR 6`	No waiting. Always costs bandwidth.

A resend takes at least one RTT (~80 ms): useful only if it arrives before its jitter-buffer slot.

FEC: repairing without asking

What Alice sends

```
Packet 5              1010 0110
Packet 6              0011 1100
Repair P = 5 XOR 6    1001 1010
```

XOR in one line

Same bits give 0, different bits give 1.

XOR the same value twice and you get the original back.

1 Send
packets 5, 6 and repair P

→

2 Network
loses packet 5

→

3 Bob computes
`P XOR 6 = 1010 0110`

→

4 Result
that is packet 5

Why use it

No NACK, no waiting a round trip. Good when the RTT is long.

Limit

One repair fixes one loss per group. 5 and 6 both lost: stuck.

Cost

Extra data always. Here 1 per 2 = +50%. Real groups: ~+10%.





Congestion: what happens in the router

Alice sends
2 Mbps

→

Router: link only 1.5 Mbps
extra 0.5 Mbps waits in a queue

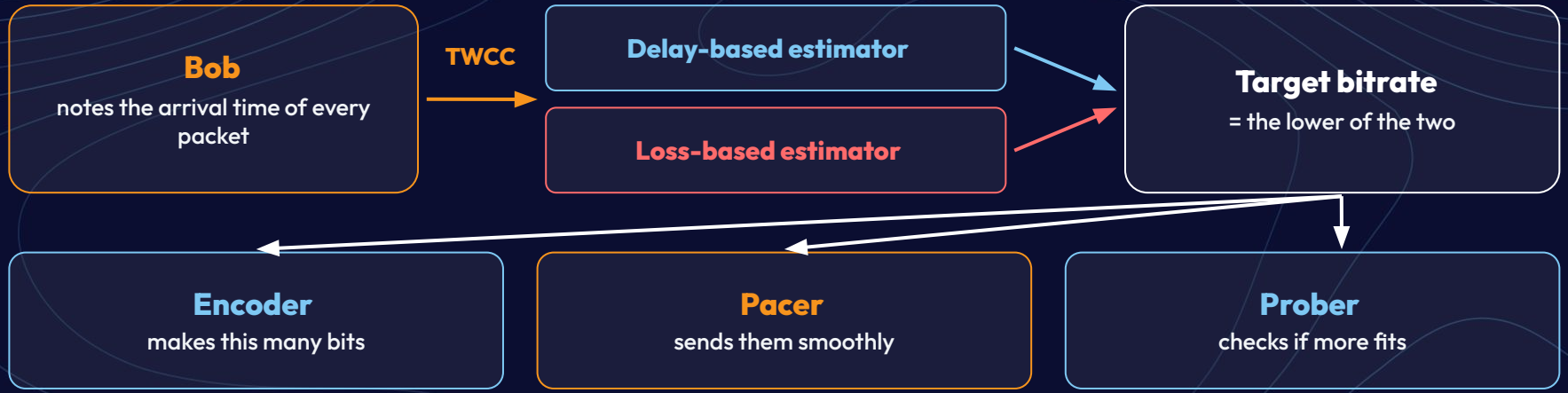
→

Bob receives
1.5 Mbps, later and later

Time	In the queue	Extra delay	What Bob notices
0.2 s	100 kbit	+67 ms	packets a bit late
0.5 s	250 kbit	+167 ms	delay keeps growing
1.0 s	queue full	+333 ms	packets dropped

Delay grows first, loss comes later. And the 1.5 Mbps is not fixed:
someone at home starts a download and the free bandwidth changes mid-call. Keep measuring.

WebRTC congestion control: the big picture



One loop, running all call long

Bob reports arrivals every ~50-100 ms. Alice's two estimators update, the target changes, and the encoder, pacer and prober follow. Algorithm: Google Congestion Control (GCC).

Delay-based estimator

Packet	Sent gap	Arrival gap	Waited extra
1-2	5 ms	5 ms	0 ms
2-3	5 ms	6 ms	+1 ms
3-4	5 ms	7 ms	+3 ms
4-5	5 ms	8 ms	+6 ms

Reading it

Packets are spaced further apart on arrival than when sent.

A queue is filling somewhere: the delay trend is rising.

Overuse	delay trend rising	cut: 0.85 x what Bob receives
Normal	delay steady	grow: up to ~8% per second
Underuse	delay falling	hold while the queue drains



Loss-based estimator

Loss reported	Rule	Estimate was 2.0 Mbps	Meaning
under 2%	x 1.05	2.1 Mbps	loss is normal noise: try more
2% to 10%	keep	2.0 Mbps	unclear: wait
over 10%	x (1 - loss / 2)	20% loss: 1.8 Mbps	real congestion: back off

Why both estimators

Delay catches queues early.

Loss catches what delay misses, e.g. routers that drop instead of queue.

Final target

Target = the lower of delay-based and loss-based.

Delay says 1.2, loss says 2.1: send 1.2 Mbps.





Probing: is there room for more?

The problem

Estimators only learn from what is sent. Sending 1 Mbps, we never learn the path could carry 3.

At call start we know nothing.

A probe

A short burst (a "cluster") at a chosen rate, sent through the pacer.

Filled with padding or old packets resent: no new media.

	Sent	Arrived over	Measured rate
Room on the path	6 x 1250 B in 30 ms = 2 Mbps	30 ms	2.0 Mbps: estimate up
Path is full	6 x 1250 B in 30 ms = 2 Mbps	50 ms	1.2 Mbps: that is the limit

When: right at call start (e.g. at 3x and 6x the start rate), after the rate dropped and conditions improved, and when the app sends little but may need more.



The pacer: smooth sending

Encoder
keyframe: 40 packets at once

→

Pacer queue
packets wait here, sorted by priority

→

Every 5 ms
send a budget of bytes

→

Network
smooth, no bursts

The budget, with numbers

```
target 1 Mbps, pacing rate 2.5 x = 2.5 Mbps
per 5 ms: 2.5 Mbps x 5 ms = 1560 B = about 1 packet
keyframe 40 x 1200 B: ~150 ms instead of 1 ms
```

Who leaves first

1 audio 2 resent packets
3 video 4 padding, probes

Why it matters

Without it, 40 packets hit the router in 1 ms: the queue overflows, loss and jitter follow, even though the average rate was fine. With it, the router sees a steady flow, and the estimators see clean measurements.





Resolving congestion: slower or less

Send slower

Same data, spread over more time.

Fine when there is no deadline: file transfer, chat.

TCP does this for you inside the operating system.

Send less

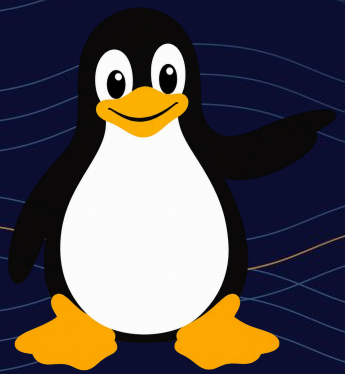
Same deadline, less data: lower video quality or resolution.

The only option for live media: frames cannot wait.

Needs a tight loop between encoder and controller.

Estimate drops to 800 kbps → **Encoder lowers bitrate** → **Video a bit softer, still live**

Part 4: Media and data communication



The problem: carrying media and data

The situation

The path is connected, secure and congestion-controlled.

Live audio and video are huge, fragile and must stay on time.

Apps also need chat, files, game state.

What we need

A packet format that labels media: RTP.

Feedback from the receiver: RTCP.

Messages with a chosen reliability: SCTP data channels.

Our plan

Media: what we get, compression, frames, RTP header and fields, RTCP messages. Data: data channels, SCTP and DCEP, opening a channel, delivering a message, channel types.



What WebRTC media gives you

Any number of streams

Add or remove them during the call.

Bundle them: screen share + webcam + microphone.

Codec agnostic

RTP can carry any codec, even future ones.

Both sides must support it: agreed in the SDP.

Adapts

Follows bandwidth and loss changes, using the congestion control of Part 3.

RTP carries the media

Streams, timing and ordering for the media pipeline.

No rules on latency or reliability: it gives the tools to build them.

RTCP carries metadata

Statistics about the call, loss handling, congestion feedback.

Flows both ways so each side can react.



Video is huge: compression

Why compress? 30 minutes of raw 720p video, 30 frames per second:

```
1280 x 720 pixels x 3 bytes      = 2.76 MB per frame
2.76 MB x 30 frames per second   = 83 MB/s  (664 Mbps)
83 MB/s x 1800 s                 = about 150 GB
```

Lossless or lossy

Lossless keeps every detail: too big, too slow.

Lossy drops what the eye barely sees. RTP media is lossy.

Intra-frame: inside one picture

Squeeze a single frame on its own, like saving a photo as JPEG.

Example: a blue sky is "this whole area is blue", not 10,000 pixels.

Inter-frame: between pictures

Send only what changed since the last frame.

Example: a talking head: send the moving mouth, "the rest is the same".

Result: 663 Mbps becomes about 1-2 Mbps. Opus for audio; VP8, H.264 (required), VP9, AV1 for video.



Video frames: I, P and B



One lost piece: every frame that depends on it is broken until the next I-frame.

I-frame

A complete picture. Decodes on its own. Big.

P-frame

Only changes from the previous picture. Small.

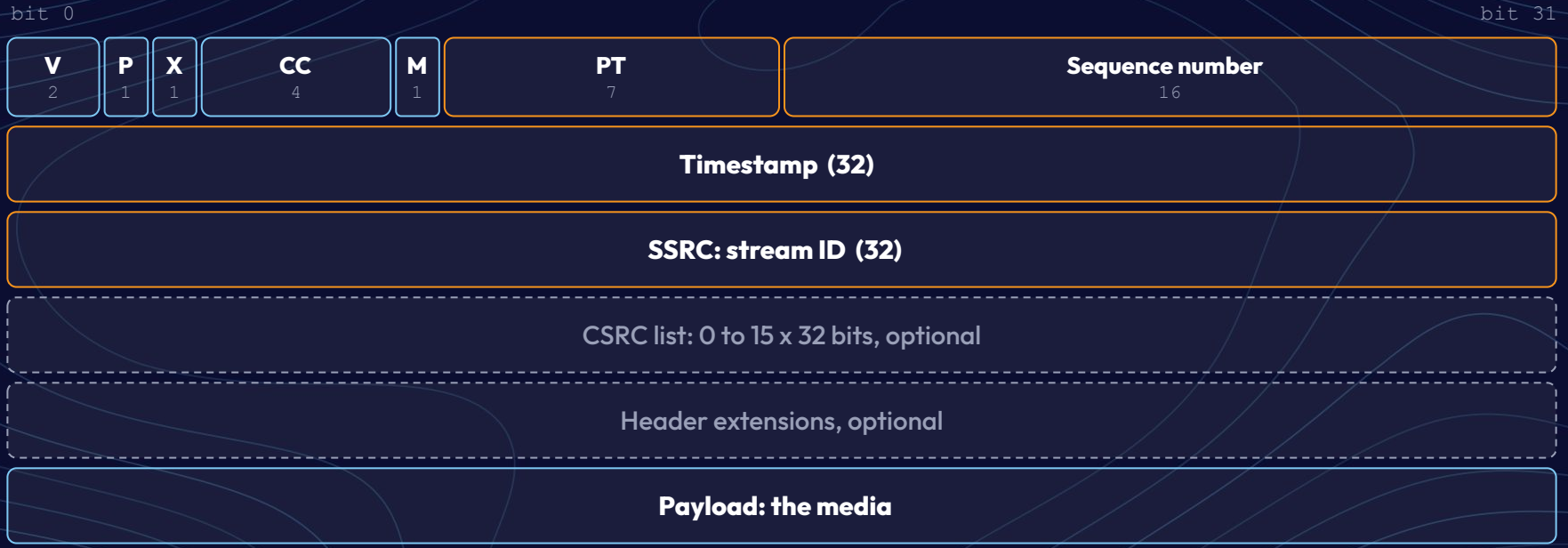
B-frame

Changes from previous and future pictures. Needs waiting, so real-time WebRTC usually avoids it.

Video is delicate

Compression is stateful: frames build on each other. The receiver must ask for a fresh I-frame (PLI, in RTCP).

RTP: the packet header



Fixed part: the first 3 rows = 12 bytes. The orange fields do the main work.

RTP header fields explained

Field	Bits	What it does	Example (Opus)
V Version	2	Always 2	2
P Padding	1	Payload ends with padding; last byte says how much	0
X Extension	1	Header extensions follow	1
CC CSRC count	4	How many CSRCs follow	0
M Marker	1	Video: last packet of a frame. Audio: talk starts	0
PT Payload type	7	Which codec. Numbers mapped in the SDP	111 = Opus
Sequence number	16	+1 per packet: order and loss detection	1002
Timestamp	32	Media clock when sampled. One video frame = one value	49920
SSRC	32	Which stream this packet belongs to	0x3a8f12c4
CSRC	32 each	Streams mixed into this one, e.g. who is talking	none



RTP example, Opus audio

seq 1000	seq 1001	seq 1002	seq 1003	seq 1004
ts 48000	ts 48960	ts 49920		ts 51840
20 ms of audio	20 ms of audio	20 ms of audio	lost	20 ms of audio

Every 20 ms

50 packets per second.

Timestamp +960 each (48 kHz clock).

Gap in seq

The receiver sees 1003 is missing.

It can conceal or ask for a resend.



RTCP: the feedback channel

bit 0 bit 31

V 2	P 1	RC / FMT 5	PT: packet type 8	Length 16
Payload: depends on the packet type. Several RTCP packets can be stacked in one UDP packet.				

PT	Name	Carries
200	**Sender Report**	what I sent: packet and byte counts, timestamps
201	**Receiver Report**	what I got: loss, jitter, timing for RTT
205	**Generic RTP feedback**	NACK (resend), TWCC (arrival times)
206	**Payload-specific feedback**	PLI and FIR (send a keyframe), REMB

RC: number of reports, or FMT: which feedback message inside type 205/206.

RTCP messages in action

What happened	Bob sends	Alice does
Part of a frame lost, decoder stuck	`PLI for SSRC 0x3a8f12c4`	encoder sends a new keyframe
Carol just joined the call	`FIR`	new keyframe, so Carol can start
Packet 1003 missing	`NACK SSRC + seq 1003`	resends 1003, if it still has it
About every second	`Receiver Report`	reads loss, jitter, RTT for stats and control

PLI vs FIR

PLI: frames lost. FIR: someone new. Both: "send a keyframe".

NACK is cheap

Asks for one small packet, not a whole frame. Old packets are ignored.





Data channels: what you get

Up to 65,534 channels

Between two peers, each with its own settings.

Messages, not bytes

Each send arrives as one whole message. Default: reliable and in order.

UDP-like if you want

Unordered and lossy for low latency, e.g. game state.

```
const ch = pc.createDataChannel("cursor", {
    ordered: false, maxRetransmits: 0 });
ch.send(JSON.stringify({ x: 120, y: 48 }));
```

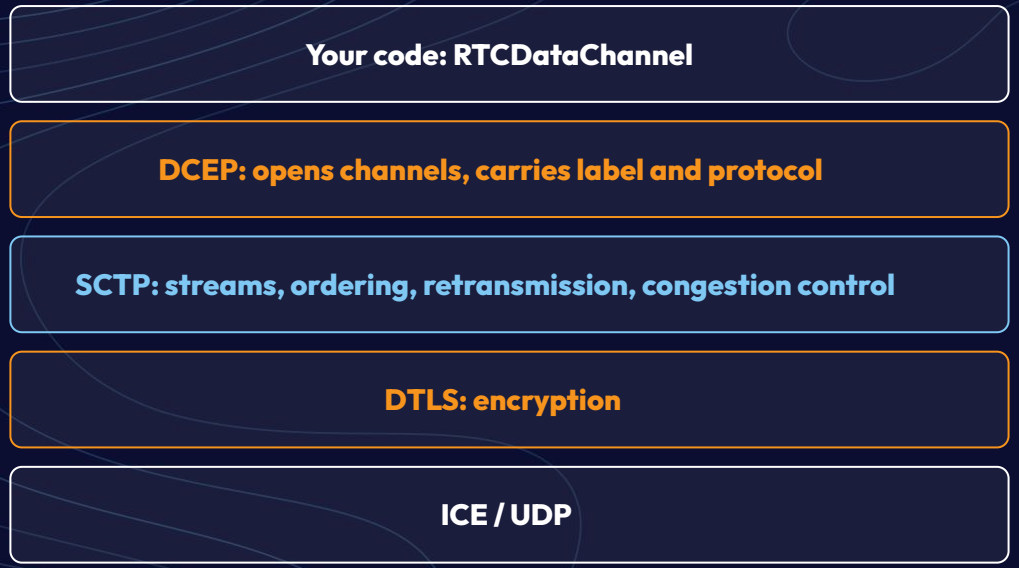
Why not WebSocket?

Peer to peer, no server. UDP-like option. Backpressure: bufferedAmount.

Runs on the same secure connection as the media: no extra handshake, no extra NAT traversal.



How data channels work: SCTP and DCEP



How the pieces fit

One data channel = one SCTP stream.

Its ordering and reliability settings go straight to SCTP.

SCTP has no place for a channel name, so DCEP adds the label and protocol.

SCTP runs inside DTLS, not directly on IP.

SCTP: RFC 4960, built as an alternative to TCP and UDP. DCEP: RFC 8832.

Opening a channel: DCEP

Alice runs createDataChannel("cursor", { ordered: false, maxRetransmits: 0 }). On the wire:

DATA_CHANNEL_OPEN	Value	Meaning
Message type	`0x03`	this is an OPEN
Channel type	`0x81`	partial reliable, retransmit limit, unordered
Priority	`256`	normal; higher is scheduled first
Reliability parameter	`0`	retransmissions allowed (or ms, if timed)
Label	`"cursor"`	channel name, UTF-8
Protocol	`""`	optional subprotocol name

Bob answers DATA_CHANNEL_ACK

`message type 0x02. Channel is open.`

Only two messages

OPEN and ACK. Sent on the new stream, PPID 50.





How SCTP and DCEP deliver a message

1 Open
DCEP gives a new SCTP stream a name and rules

2 Send
ch.send("hi"): SCTP packs it into numbered chunks

3 Travel
inside DTLS, over the same UDP path as media

4 Check
SCTP acks what arrived, resends or gives up per the rules

5 Deliver
the whole message, on the right channel: onmessage

DCEP: once per channel

Runs when a channel opens: OPEN, then ACK.

Says which stream is which channel, and how reliable it must be.

After that it is silent.

SCTP: every message

Splits big messages, numbers the pieces, acknowledges them.

Resends lost pieces, or skips them on lossy channels.

Puts them back in order if the channel is ordered.





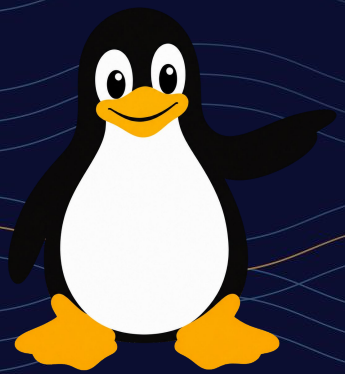
Six channel types

Channel type	Can a message be lost?	Order
Reliable	Never	In order
Reliable, unordered	Never	Any order
Retransmit limit	After N retries	In order
Retransmit limit, unordered	After N retries	Any order
Timed	After a time limit	In order
Timed, unordered	After a time limit	Any order





Part 5: Topologies and practice



The problem: more than two people

The situation

Two people is easy. A 20-person meeting is not.

Sending 19 copies of your video overloads your uplink.

What we need

A server that shares the work.

Different quality for different viewers.

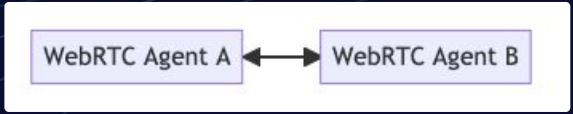
Tools to see what went wrong.

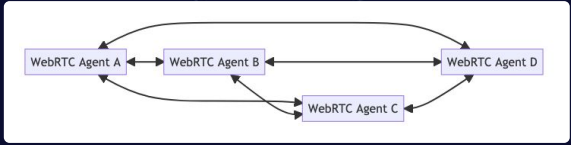
Our plan

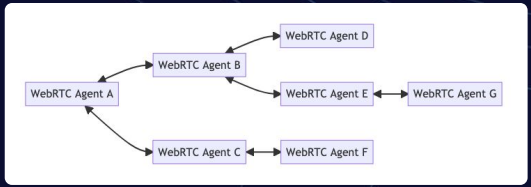
1 P2P and meshes 2 SFU and MCU 3 Simulcast and SVC Then: the whole picture



P2P, full mesh, hybrid mesh







P2P

Two peers, one link.

The simple case.

Full mesh

Everyone links to everyone.

Fine for 2 to 4 people.

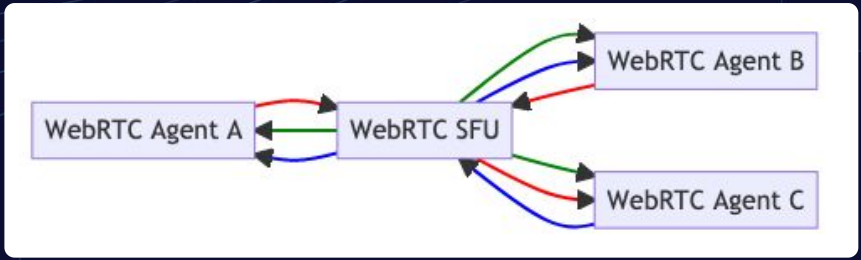
Hybrid mesh

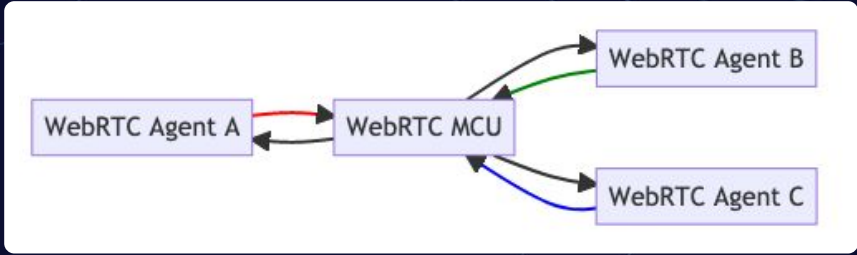
Not everyone links to everyone.

Some peers relay for others.

Full mesh: 5 people = 10 connections, 20 people = 190.

SFU and MCU





SFU

Everyone sends one stream up.

The server forwards packets without decoding. Most common.

MCU

The server decodes and mixes all streams into one per client.

Light for clients, heavy on server CPU.

Arrow colours are different streams. An SFU routes packets; it does not decode.

Simulcast and SVC

Simulcast

Sender sends several versions, e.g. 3 resolutions.

SFU forwards one to each receiver.

SVC

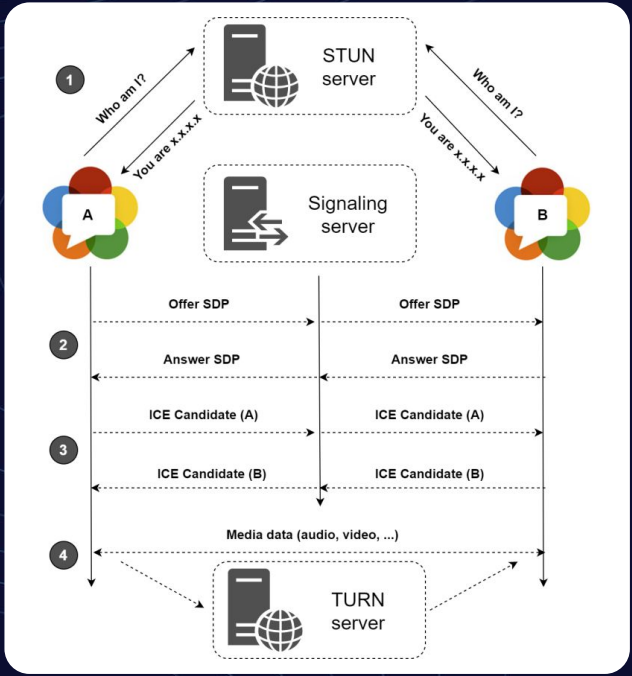
One stream built from layers.

SFU drops upper layers for slow receivers.

The SFU adapts per receiver without re-encoding anything.



The whole picture, with servers



1. STUN: each peer asks "who am I?" and learns its public address.

2. Offer and answer SDP travel through the signaling server.

3. ICE candidates are trickled both ways through signaling.

4. Media flows peer to peer, or through TURN if there is no direct path.

Questions

?

